Ryerson University

Department of Electrical, Computer, & Biomedical Engineering
Faculty of Engineering & Architectural Science

Course Title:	Distributed and Cloud Computing
Course Number:	COE 892
Semester/Year (e.g.F2016)	w2026

Instructor:	Dr. Muhammad Jaseemuddin

Assignment/Lab Number:	2
Assignment/Lab Title:	gRPC

Submission Date:	Mar 6, 2026
Due Date:	Mar 6, 2026

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Pathirana	Chanuth	501107354	01	C.P

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: http://www.ryerson.ca/senate/current/pol60.pdf

Introduction

The objective of this lab is to build on the previous lab (Lab 1) by creating two programs using gRPC to simulate the communication of several rovers with ground control. This is achieved by creating a .proto file with 5 specific methods, creating a server that uses the map.txt file from Lab 1, command files, and a mine serial number, and finally creating a client side that implements the 5 methods in the .proto file and must accept an argument for the rover ID (1-10). It is important to note that the work of each rover is handled independently, meaning that upon reaching a mine, a rover must disarm the mine irrespective of it being disarmed previously by another rover.

Implementation

Part 1: Create the Proto Buffer Files

As stated in the lab manual, the .proto file defines the structure required for gRPC communication between the rovers (clients) and the ground control (server). The service defined is RoverControl, which includes five RPC methods used for the communication. These methods are GetMap(), which allows a rover to retrieve the map (map1.txt), GetCommands(), which streams the rover's movement commands from the server, and GetMineSerial(), which returns the serial number of a mine when a rover encounters one. The rover then sends the computed PIN to the server using SubmitMinePin(). Finally, ReportExecutionStatus() allows the rover to notify the server whether all commands were executed successfully. Each RPC uses specific request and response messages such as MapRequest/MapResponse, CommandRequest/Command, MineSerialRequest/MineSerialResponse, MinePinRequest/MinePinResponse, and ExecutionStatusRequest/ExecutionStatusResponse. These messages define the data exchanged between the client and server during the rover's operation.

```
syntax = "proto3";

package rover;

// Map cell and map as 2D grid (from map1.txt)
message MapCell {
 int32 row = 1;
 int32 col = 2;
 int32 value = 3; // 0 = empty, 1 = mine
}

message MapData {
 int32 rows = 1;
 int32 cols = 2;
 repeated int32 data = 3; // row-major flattened grid (size = rows * cols)
}

// Request/response for GetMap
```

```
message MapRequest {
 int32 rover_id = 1;
}

message MapResponse {
 MapData map = 1;
}

// Command streaming (server → client)
message CommandRequest {
 int32 rover_id = 1;
}

// Each command is one character: 'L','R','M','D'
message Command {
 string cmd = 1;
}

// Mine serial number (server → client)
message MineSerialRequest {
 int32 rover_id = 1;
 int32 row = 2;
 int32 col = 3;
}

message MineSerialResponse {
 string serial = 1; // from mines.txt
}

// Mine PIN (client → server)
message MinePinRequest {
 int32 rover_id = 1;
 string serial = 2;
 int64 pin = 3;
}

message MinePinResponse {
 bool accepted = 1;
}

// Rover execution status (client → server)
message ExecutionStatusRequest {
 int32 rover_id = 1;
 bool success = 2;
 string message = 3;
}

message ExecutionStatusResponse {
 bool ack = 1;
```

```
}

// The service definition
service RoverControl {
 // 1. Get map
 rpc GetMap(MapRequest) returns (MapResponse);

 // 2. Get a stream of commands (server-streaming)
 rpc GetCommands(CommandRequest) returns (stream Command);

 // 3. Get a mine serial number
 rpc GetMineSerial(MineSerialRequest) returns (MineSerialResponse);

 // 4. Notify if all commands executed successfully
 rpc ReportExecutionStatus(ExecutionStatusRequest) returns
(ExecutionStatusResponse);

 // 5. Share a mine PIN with the server
 rpc SubmitMinePin(MinePinRequest) returns (MinePinResponse);
}
```

Part 2: Create the Server

Once the .proto file is defined, the gRPC server is created to act as the ground control. The server loads the map file (map1.txt) and mine serial numbers (mines.txt), and prepares them for the rovers. The server implements the RPC methods defined in the .proto file. The GetMap() method reads the map file and sends the 2D map to the rover as a flattened gRPC message. GetCommands() generates a sequence of movement commands for each rover and streams them to the client. When a rover encounters a mine, it calls GetMineSerial(), which returns the mine's serial number based on its position. After the rover calculates the correct PIN, it sends it to the server using SubmitMinePin(), where the server verifies the PIN using a SHA-256 hash check. Finally, ReportExecutionStatus() receives and prints the rover's final execution result, indicating whether the rover completed its commands successfully.

```
import grpc
from concurrent import futures
import time
import copy
import hashlib

import rover_pb2
import rover_pb2_grpc

# ---------- MAP / MINES REUSE FROM LAB 1 ----------

def read_map(filename="map1.txt"):
   with open(filename, "r") as f:
       first_line = f.readline().strip().split()
```

```
        rows, cols = int(first_line[0]), int(first_line[1])
        land = []
        for _ in range(rows):
            land.append([int(x) for x in f.readline().strip().split()])
    return rows, cols, land

ROWS, COLS, BASE_MAP = read_map("map1.txt")

def load_mines(filename="mines.txt"):
    mines = []
    with open(filename, "r") as f:
        for line in f:
            serial = line.strip()
            if serial:
                mines.append(serial)
    return mines

MINES = load_mines("mines.txt")

def get_mine_positions(land):
    positions = []
    for r in range(len(land)):
        for c in range(len(land[0])):
            if land[r][c] == 1:
                positions.append((r, c))
    return positions

MINE_POSITIONS = get_mine_positions(BASE_MAP)

MINE_SERIAL_MAP = {}
for idx, (r, c) in enumerate(MINE_POSITIONS):
    if idx < len(MINES):
        MINE_SERIAL_MAP[(r, c)] = MINES[idx]
    else:
        MINE_SERIAL_MAP[(r, c)] = f"EXTRA-{idx}"

# ---------- COMMANDS FOR EACH ROVER ----------

def load_rover_commands(rover_id: int) -> str:
    import random
    random.seed(rover_id)
    commands = ["L", "R", "M", "D"]
    seq = "".join(random.choice(commands) for _ in range(20))
    return seq

# ---------- VERIFY PIN FUNCTION ----------

def verify_pin(pin, serial):
    temp_key = str(pin) + serial
```

```
    h = hashlib.sha256(temp_key.encode()).hexdigest()
    return h.startswith("000000")

# ---------- SERVICE IMPLEMENTATION ----------

class RoverControlServicer(rover_pb2_grpc.RoverControlServicer):
    def GetMap(self, request, context):
        land = copy.deepcopy(BASE_MAP)
        flat = []
        for r in range(ROWS):
            for c in range(COLS):
                flat.append(land[r][c])
        map_data = rover_pb2.MapData(
            rows=ROWS,
            cols=COLS,
            data=flat
        )
        return rover_pb2.MapResponse(map=map_data)

    def GetCommands(self, request, context):
        rover_id = request.rover_id
        sequence = load_rover_commands(rover_id)
        for ch in sequence:
            yield rover_pb2.Command(cmd=ch)

    def GetMineSerial(self, request, context):
        key = (request.row, request.col)
        serial = MINE_SERIAL_MAP.get(key, "")
        return rover_pb2.MineSerialResponse(serial=serial)

    def ReportExecutionStatus(self, request, context):
        rover_id = request.rover_id
        success = request.success
        msg = request.message
        print(f"[SERVER] Rover {rover_id} finished. Success={success},
msg={msg}")
        return rover_pb2.ExecutionStatusResponse(ack=True)

    def SubmitMinePin(self, request, context):
        rover_id = request.rover_id
        serial = request.serial
        pin = request.pin

        print(f"[SERVER] Rover {rover_id} submitted PIN {pin} for mine
{serial}")

        valid = verify_pin(pin, serial)

        if valid:
```

```
            print(f"[SERVER] PIN verified correct!")
        else:
            print(f"[SERVER] PIN invalid!")

        return rover_pb2.MinePinResponse(accepted=valid)

# ---------- SERVER BOOTSTRAP ----------

def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    rover_pb2_grpc.add_RoverControlServicer_to_server(
        RoverControlServicer(), server
    )
    server.add_insecure_port("[::]:50051")
    server.start()
    print("RoverControl gRPC server running on port 50051...")
    try:
        while True:
            time.sleep(86400)
    except KeyboardInterrupt:
        server.stop(0)

if __name__ == "__main__":
    serve()
```

Part 3: Create the Client Side

Both the .proto files and the server were implemented first, followed by the client program. As specified in the lab manual, the gRPC client represents the rover and accepts a CLI argument indicating the rover ID when executed. The client connects to the server and calls the remote procedures defined in the .proto file using a gRPC stub. When the client starts, it calls GetMap() to retrieve the 2D map from the server and stores it in a suitable data structure. It then calls GetCommands(), which streams the movement commands that the rover executes sequentially to update its position and direction. If the rover moves onto a cell containing a mine, the client calls GetMineSerial() to retrieve the mine's serial number. The rover then computes the disarming PIN using a brute-force SHA-256 search and sends the result to the server using SubmitMinePin(). After executing all commands, the rover reports the final result to the server using ReportExecutionStatus(), indicating whether the mission completed successfully.

```
import grpc
import sys
import copy
import hashlib

import rover_pb2
import rover_pb2_grpc

# Directions and movement (from Lab 1)
```

```
DIRECTIONS = ["N", "E", "S", "W"]
MOVE = {
   "N": (-1, 0),
   "E": (0, 1),
   "S": (1, 0),
   "W": (0, -1),
}

def turn_left(d):
   return DIRECTIONS[(DIRECTIONS.index(d) - 1) % 4]

def turn_right(d):
   return DIRECTIONS[(DIRECTIONS.index(d) + 1) % 4]

def build_map_from_response(map_resp):
   rows = map_resp.map.rows
   cols = map_resp.map.cols
   flat = list(map_resp.map.data)
   land = []
   idx = 0
   for r in range(rows):
       row = []
       for c in range(cols):
           row.append(flat[idx])
           idx += 1
       land.append(row)
   return rows, cols, land

# YOUR disarm_mine from part2.py
def disarm_mine(serial):
   pin = 0
   while True:
       temp_key = str(pin) + serial
       h = hashlib.sha256(temp_key.encode()).hexdigest()
       if h.startswith("000000"):
           print(f"[ROVER] Disarmed mine {serial} with PIN {pin}")
           return pin, h
       pin += 1

def main():
   if len(sys.argv) != 2:
       print("Usage: python3 client_complete.py <rover_id>")
       sys.exit(1)

   rover_id = int(sys.argv[1])

   channel = grpc.insecure_channel("localhost:50051")
   stub = rover_pb2_grpc.RoverControlStub(channel)
```

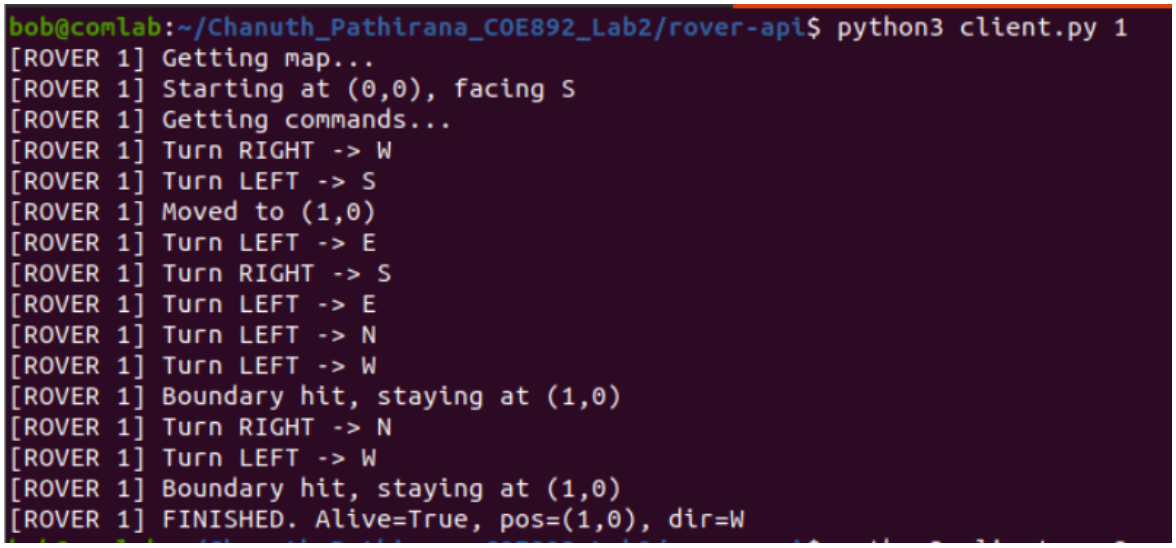
```
    # 1. Get map
    print(f"[ROVER {rover_id}] Getting map...")
    map_resp = stub.GetMap(rover_pb2.MapRequest(rover_id=rover_id))
    ROWS, COLS, land = build_map_from_response(map_resp)
    land = copy.deepcopy(land)

    # Initial state
    x, y = 0, 0
    direction = "S"
    alive = True
    last_command = None

    print(f"[ROVER {rover_id}] Starting at ({x},{y}), facing {direction}")

    # 2. Get command stream
    print(f"[ROVER {rover_id}] Getting commands...")
    cmd_stream = stub.GetCommands(rover_pb2.CommandRequest(rover_id=rover_id))

    for cmd_msg in cmd_stream:
        cmd = cmd_msg.cmd

        if not alive:
            break

        if cmd == "L":
            direction = turn_left(direction)
            print(f"[ROVER {rover_id}] Turn LEFT -> {direction}")

        elif cmd == "R":
            direction = turn_right(direction)
            print(f"[ROVER {rover_id}] Turn RIGHT -> {direction}")

        elif cmd == "D":
            if land[x][y] == 1:
                land[x][y] = 0
                print(f"[ROVER {rover_id}] Digged current cell ({x},{y})")

        elif cmd == "M":
            dx, dy = MOVE[direction]
            nx, ny = x + dx, y + dy

            if nx < 0 or nx >= ROWS or ny < 0 or ny >= COLS:
                print(f"[ROVER {rover_id}] Boundary hit, staying at ({x},{y})")
                continue

            if land[nx][ny] == 1:
                print(f"[ROVER {rover_id}] Mine detected at ({nx},{ny})")

                # Get serial
```

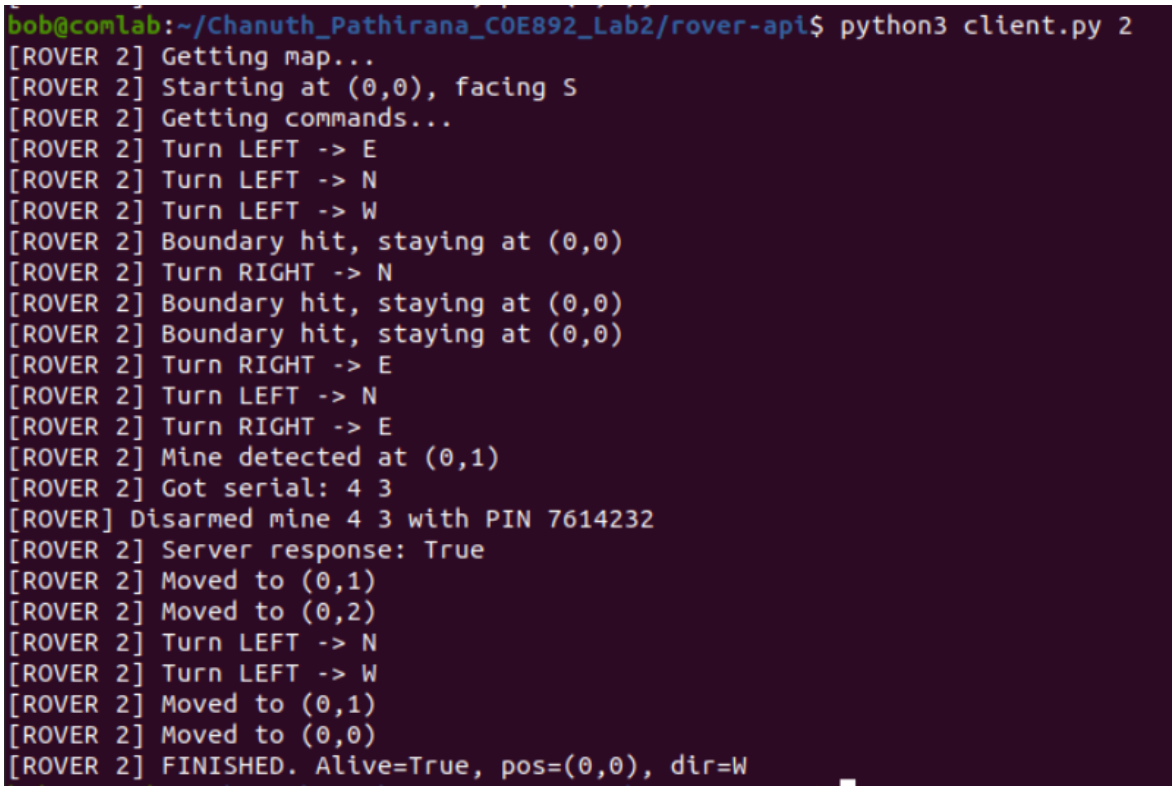
```
                ms_req = rover_pb2.MineSerialRequest(rover_id=rover_id, row=nx,
col=ny)
                ms_resp = stub.GetMineSerial(ms_req)
                serial = ms_resp.serial
                print(f"[ROVER {rover_id}] Got serial: {serial}")

                # BRUTE-FORCE YOUR PIN
                pin, hash_value = disarm_mine(serial)

                # Submit to server
                pin_resp = stub.SubmitMinePin(
                    rover_pb2.MinePinRequest(rover_id=rover_id, serial=serial,
pin=pin)
                )
                print(f"[ROVER {rover_id}] Server response:
{pin_resp.accepted}")

                land[nx][ny] = 0

            x, y = nx, ny
            print(f"[ROVER {rover_id}] Moved to ({x},{y})")

        last_command = cmd

    # 4. Report status
    status_req = rover_pb2.ExecutionStatusRequest(
        rover_id=rover_id,
        success=alive,
        message="All commands executed" if alive else "Destroyed by mine"
    )
    stub.ReportExecutionStatus(status_req)

    print(f"[ROVER {rover_id}] FINISHED. Alive={alive}, pos=({x},{y}),
dir={direction}")

if __name__ == "__main__":
    main()
```

Results

```
bob@comlab:~/Chanuth_Pathirana_COE892_Lab2/rover-api$ python3 server.py
RoverControl gRPC server running on port 50051...
[SERVER] Rover 1 finished. Success=True, msg=All commands executed
[SERVER] Rover 2 submitted PIN 7614232 for mine 4 3
[SERVER] PIN verified correct!
[SERVER] Rover 2 finished. Success=True, msg=All commands executed
```

The server.py implements the gRPC ground control service on port 50051, loading map1.txt and mines.txt data. It serves terrain maps via GetMap, streams rover-specific commands via GetCommands, provides mine serials via GetMineSerial, verifies submitted PINs using SHA-256 six-zero-prefix validation from Lab 1, and logs execution status from ReportExecutionStatus. Console output shows successful rover completions and PIN verifications.





The client.py accepts a rover ID via CLI argument and connects to the server at localhost:50051. It retrieves the map via GetMap, executes streamed commands (L/R/M/D) from GetCommands using Lab 1 movement logic, handles boundary checks, and when encountering

mines calls GetMineSerial then brute-forces valid PINs using Lab 1's disarm_mine function before submitting via SubmitMinePin. Finally reports status via ReportExecutionStatus. Console logs show position, direction, and mine disarming.

Conclusion

In conclusion, the client and server were able to successfully communicate with each other, which is also attributed to the creation and handling of the .proto files. Overall, this is a great learning experience in how cloud computing is crucial for communication that will be beneficial not just for Lab 3, but for future and revolutionary applications.